COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE 010	SECTION: CPE21S4	
DATE PERFORMED:	7/08/2026	DATE SUBMITTED:	7/08/2026
NAME:	Jullian Edrain R. Sevilla	INSTRUCTOR:	Engr. JIMLORD QUEJADO

ACTIVITY NO. ___ [ACTIVITY NAME]

1. PROCEDURE OUTPUT

What procedures did you perform?

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: Copilot

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Code assistance

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: Help me code the procedure and supplementary into “Programiz” because my VS Code is still not working

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.

main.cpp

```cpp
#include <iostream>
#include <string>
using namespace std;

class Queue {
private:
    int front, back, size, capacity;
    string* q_array;

public:
    Queue(int cap) {
        capacity = cap;
        q_array = new string[capacity];
        front = 0;
        back = -1;
        size = 0;
    }

    ~Queue() {
        delete[] q_array;
    }

    bool Empty() { return size == 0; }
    bool Full() { return size == capacity; }

    void Enqueue(string name) {
        if (Full()) {
            cout << "Queue is full!\n";
            return;
        }
        back = (back + 1) % capacity;
        q_array[back] = name;
        size++;
        cout << "Enqueued: " << name << endl;
    }

```

main.cpp

```
    void Dequeue() {
        if (Empty()) {
            cout << "Queue is empty!\n";
            return;
        }
        cout << "Dequeued: " << q_array[front] << endl;
        front = (front + 1) % capacity;
        size--;
    }

    void Display() {
        if (Empty()) {
            cout << "Queue is empty!\n";
            return;
        }
        cout << "Queue contents: ";
        for (int i = 0; i < size; i++) {
            cout << q_array[(front + i) % capacity] << " ";
        }
        cout << endl;
    }
};

int main() {
    Queue q(5);

    q.Enqueue("Jullian");
    q.Enqueue("Mark");
    q.Enqueue("Ella");
    q.Display();

    q.Dequeue();
    q.Display();

    q.Enqueue("Sofia");
    q.Enqueue("Liam");
```

```
73      q.Enqueue("Noah");
74      q.Display();
75
76      return 0;
77  }
78
```

Output

```
Enqueued: Jullian
Enqueued: Mark
Enqueued: Ella
Queue contents: Jullian Mark Ella
Dequeued: Jullian
Queue contents: Mark Ella
Enqueued: Sofia
Enqueued: Liam
Enqueued: Noah
Queue contents: Mark Ella Sofia Liam Noah


=== Code Execution Successful ===
```

Supplementary:

main.cpp

Share Run

```cpp
#include <iostream>
#include <string>
using namespace std;

// Job class
class Job {
public:
    int jobID;
    string userName;
    int pages;

    Job() {}
    Job(int id, string user, int p) {
        jobID = id;
        userName = user;
        pages = p;
    }

    void displayJob() {
        cout << "Job ID: " << jobID << ", User: " << userName << ", Pages: " <<
            pages << endl;
    }
};

// Printer class (array-based queue)
class Printer {
private:
    Job* queue;
    int front, back, size, capacity;

public:
    Printer(int cap) {
        capacity = cap;
        queue = new Job[capacity];
        front = 0;
        back = -1;
```

main.cpp

Share Run

```
36          size = 0;
37      }
38
39      ~Printer() {
40          delete[] queue;
41      }
42
43      bool isEmpty() { return size == 0; }
44      bool isFull() { return size == capacity; }
45
46      void addJob(Job j) {
47          if (isFull()) {
48              cout << "Printer queue is full! Cannot add job.\n";
49              return;
50          }
51          back = (back + 1) % capacity;
52          queue[back] = j;
53          size++;
54          cout << "Added job: ";
55          j.displayJob();
56      }
57
58      void processJob() {
59          if (isEmpty()) {
60              cout << "No jobs to process.\n";
61              return;
62          }
63          cout << "Processing job: ";
64          queue[front].displayJob();
65          front = (front + 1) % capacity;
66          size--;
67      }
68
69      void showQueue() {
70          if (isEmpty()) {
71              cout << "Printer queue is empty.\n";
```

main.cpp

Share Run

```
            return;
        }
        cout << "\nPending Jobs:\n";
        for (int i = 0; i < size; i++) {
            queue[(front + i) % capacity].displayJob();
        }
        cout << endl;
    }
};

// Main simulation
int main() {
    Printer printer(5);

    printer.addJob(Job(1, "Jullian", 3));
    printer.addJob(Job(2, "Ella", 5));
    printer.addJob(Job(3, "Mark", 2));

    printer.showQueue();

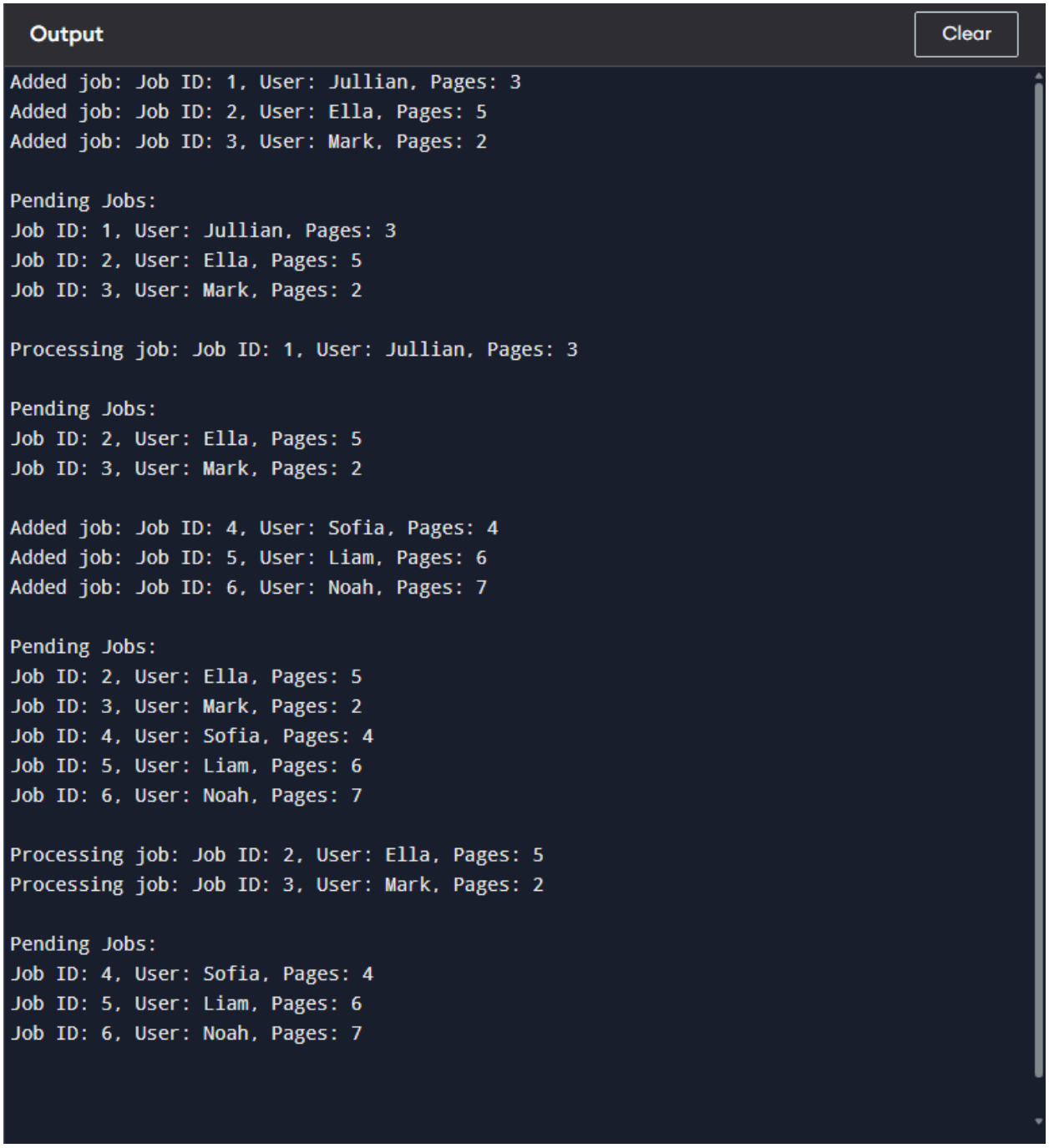
    printer.processJob();
    printer.showQueue();

    printer.addJob(Job(4, "Sofia", 4));
    printer.addJob(Job(5, "Liam", 6));
    printer.addJob(Job(6, "Noah", 7));

    printer.showQueue();

    printer.processJob();
    printer.processJob();
    printer.showQueue();

    return 0;
}
```



Conclusions/Learnings

I learned that a queue works like a line, first in, first out. The printer simulation showed how jobs go one by one. Writing the program helped me see how arrays can store jobs and how they move as they're processed. It was hard at first, but I got it working. I think I did okay, but I can still improve by learning more about linked lists.